KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 03

Laboratuvar Föyü

Dört temel ilke: tespit · kapsülleme onarımı · miras · çok biçimlilik

ÖZET

Bu laboratuvar oturumunda önce verilen bir kod parçasında dört ilkenin hangi satırlarda kullanıldığını **tespit edecek**, ardından kapsülleme ihlali içeren bir sınıfı onaracak, iki seviyeli bir miras hiyerarşisi kuracak, çok biçimli bir dizi üzerinde tek döngüyle işlem yapacak ve son olarak dört ilkeyi birlikte kullanan mini bir kütüphane sistemi yazacaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 3 ders notu Bölüm 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta03_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- Verilen bir kod parçasında kapsülleme, soyutlama, miras ve çok biçimlilik ilkelerinin hangi satırlarda kullanıldığını tespit edebilir.
- Kapsülleme ihlali içeren bir sınıfı `private` alan + denetimli metot düzenine dönüştürebilir.
- `extends` ile iki seviyeli bir miras hiyerarşisi kurabilir ve `super(...)` çağrısını doğru yere yazabilir.
- Üst sınıftaki bir metodu `@Override` ile ezebilir ve ezme ile aşırı yüklemeyi ayırt edebilir.
- Üst tür referansı içeren bir dizi üzerinde tek döngüyle çok biçimli işlem yapabilir.
- `instanceof` ile güvenli aşağı dönüşüm yazabilir ve neden gerekli olduğunu açıklayabilir.
- Soyut sınıf ve arayüzü birlikte kullanarak küçük bir sistem tasarlayabilir.



Şekil 3.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

- Her sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.

- Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`
- Sınıf adları `PascalCase`, değişken ve metot adları `camelCase` olmalıdır.
- Değişken, sınıf ve metot adlarında **Türkçe karakter kullanmayın**.
- Tüm alanları `private` yapın; dışarının okuması gerekiyorsa getter yazın. Setter yazmadan önce Görev 2'deki ölçütü uygulayın.
- Ezdiğiniz her metoda `@Override` işaretleyicisini **mutlaka** yazın.
- Her dosyanın başına ad, soyad, numara ve görev numarasını yorum olarak yazın.
- Kontrol sorularının yanıtlarını ilgili dosyanın sonuna yorum bloğu olarak ekleyin.

> **DİKKAT**
>
> İnternetten ya da yapay zekâ araçlarından alınan kodun **doğrudan** teslim edilmesi kabul edilmez. Yazdığınız her satırı açıklayabilmeniz beklenir; laboratuvar sorumlusu rastgele öğrencilerden kodunu açıklamasını isteyebilir. Özellikle **hangi satırın hangi ilkeyi** gerçeklediğini sorabilir.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre: 15 dakika · Puan: değerlendirmeye katılmaz, ancak zorunludur

Amaç

Ders örneklerinin sizin bilgisayarınızda da derlenip çalıştığını doğrulamak ve bu haftanın çalışma klasörünü hazırlamak.

Görev Tanımı

1. `Hafta03` adında bir klasör açın; içinde `src` ve `bin` alt klasörleri oluşturun.
2. Ders klasöründeki `Sekil.java`, `Daire.java`, `Kare.java`, `Ucgen.java` ve `SekilDemo.java` dosyalarını `src` içine kopyalayın.
3. Tümünü derleyin: `javac -encoding UTF-8 -d bin src/*.java`
4. Çalıştırın: `java -cp bin SekilDemo`
5. Çıktının aşağıdaki ilk beş satırla birebir aynı olduğunu doğrulayın.

```
▸ TERMINAL — Beklenen çıktı (ilk beş satır)
--- 1) COK BICIMLILIK: tek dongu, uc farkli davranis ---
  Daire       renk=kirmizi   alan=   28.274  cevre=   18.850
  Kare        renk=mavi      alan=   16.000  cevre=   16.000
  Dik ucgen   renk=yesil     alan=   12.000  cevre=   17.211
  TOPLAM                             56.274
```

> **İPUCU**
>
> `Sekil.java` tek başına derlenebilir ama çalıştırılamaz: soyut sınıftan nesne üretilemediği için `main` metodu

> yoktur. Bu normaldir. Yalnızca `SekilDemo` sınıfını çalıştırın.

Kontrol Soruları

1. `java -cp bin Sekil` komutunu çalıştırırsanız ne olur? Neden?
2. `SekilDemo.java` dosyası `Daire`, `Kare` ve `Ucgen` sınıflarını `import` etmiyor. Buna rağmen neden derleniyor?

Görev 1 — İlke Tespiti

Süre: 20 dakika · Puan: 20 · Zorluk: ●○○

Amaç

Dört temel ilkeyi çalışan bir kod üzerinde tanımak. Bu görev kod yazmaktan çok **kod okumaya** dayanır; haftanın kavramsal temelini kurar.

Görev Tanımı

Size verilen `Gorev1_IlkeTespiti.java` dosyası bir oyun karakteri sisteminin küçük bir sürümüdür ve dört bölge içerir: **A**, **B**, **C**, **D**. Her bölge dört ilkeden **tam olarak birini** örnekler.

Gorev1_IlkeTespiti.java

```java
class Karakter {
  private final String ad;
  private int can;

  Karakter(String ad, int can){
    this.ad = ad;
    this.can = (can > 0) ? can : 1;
  }

  String getAd(){
    return ad;
  }

  int getCan(){
    return can;
  }

  void hasarAl(int miktar){
    if(miktar <= 0){
      System.out.println("   Uyari: hasar pozitif olmalidir; islem yapilmadi.");
      return;
    }
          can = Math.max(0, can - miktar);
  }

  String tanit(){
    return String.format("%-8s can=%3d  tur=Karakter", ad, can);
  }
}
```

Kod 3.1 — Görev 1 — verilen kodun bir bölümü

Dosyanın tamamını laboratuvar klasöründen alın, derleyin ve çalıştırın. Ardından aşağıdaki tabloyu **programı çalıştırmadan önce** doldurun; sonra çıktıdaki yanıt anahtarıyla karşılaştırın.

Bölge	Hangi ilke?	Gerekçe (hangi satır / hangi anahtar sözcük?)
A		
B		
C		
D		

Tablo 3.2 — Doldurulacak tespit tablosu

Gereksinimler

- Tabloyu doldurup dosyanın sonuna yorum bloğu olarak ekleyin.
- Her gerekçede **en az bir anahtar sözcük** (`private`, `interface`, `extends`, `@Override`) veya **satır numarası** geçmelidir.
- Ayrıca `Karakter` sınıfındaki `hasarAl()` metoduna, canı **sıfırın altına düşürmeyecek** bir koruma zaten vardır; bu korumanın hangi satırda olduğunu yazın.
- Programı çalıştırıp kendi tablonuzla yanıt anahtarını karşılaştırın; farklı düşündüğünüz bölge varsa nedenini yazın.

Beklenen Çıktı

```
▸ TERMINAL — Gorev1_IlkeTespiti — ilk bölüm
=== ILKE TESPITI ===
   Uyari: hasar pozitif olmalidir; islem yapilmadi.
A) can = 70  (dogrudan yazilamaz)
B) sozlesme uzerinden: 50 (sinif adi kullanilmadi)
C) Savasci hasarAl() yazmadi, can = 100
D) tek dongu, uc farkli davranis:
   Gezgin   can=100  tur=Karakter
   Kaan     can=120  tur=Savasci   guc=50
   Elif     can= 80  tur=Buyucu    guc=45
```

> **DİKKAT**
>
> `Savasci` ve `Buyucu` sınıflarının ikisi de `saldiriGucu()` metodunu gerçekliyor ama sonuçları farklı: `50` ve `45`. Aynı çağrının farklı sonuç üretmesi rastlantı değildir; bu, D bölgesinin ilkesidir.

Kontrol Soruları

1. B bölgesindeki `Savasabilir s = new Savasci(...)` bildiriminde, `s` referansıyla `hasarAl()` metodunu çağırabilir misiniz? Neden?

2. C bölgesinde `Savasci` sınıfı `hasarAl()` metodunu hiç yazmamış. Bu metot nereden geliyor ve `can` alanına nasıl erişebiliyor?
3. D bölgesindeki dizinin türü `Karakter[]`. Diziye bir `Buyucu` koyabilmemizin nedeni nedir?

Görev 2 — Kapsülleme İhlalini Onarma

Süre: 25 dakika · Puan: 20 · Zorluk: ●●○

Amaç

Kapsülleme ihlali içeren bir sınıfı, veriyi ve kuralı bir arada tutan doğru bir tasarıma dönüştürmek.

Görev Tanımı

Aşağıdaki `Sensor` sınıfı bir sıcaklık sensörünü temsil ediyor ve tamamen korumasız yazılmış. Bu sınıfı, ders notu Bölüm 4.1'deki **düzey 3** kapsüllemeye taşıyın.

Sensor.java

```java
// BOZUK SURUM - Gorev 2'de bunu duzelteceksiniz
public class Sensor{
  public String kod;            // herkes degistirebilir
  public String birim;
  public double deger;          // hicbir aralik denetimi yok
  public int olcumSayisi;       // disaridan istenen degere ayarlanabilir

  public Sensor(String kod, String birim){
    this.kod = kod;
    this.birim = birim;
  }
}

// Cagiran kod boyle davranabiliyor:
//   s.deger = 9999.0;         // sicaklik sensoru icin anlamsiz
//   s.olcumSayisi = 100;      // hic olcum yapilmadan sayac doldu
//   s.kod = "";               // sensorun kimligi silindi
```

Kod 3.2 — Onarılacak bozuk sürüm

Gereksinimler

#	Gereksinim	Ölçüt
1	Tüm alanları `private` yapın	Sınıf dışından hiçbir alana doğrudan erişilememeli
2	`kod` ve `birim` alanlarını `final` yapın	Doğumda belirlenmeli, sonradan değişmemeli; setter yazılmamalı
3	İki sabit tanımlayın	`public static final double EN_DUSUK = -50.0;` ve `EN_YUKSEK = 150.0;`
4	`olc(double)` metodu yazın	Değer aralık dışındaysa **reddedilmeli**, `false` dönmeli ve neden yazdırılmalı
5	Sayaçları sınıf kendisi tutsun	`olcumSayisi` ve `redSayisi` dışarıdan **artırılamamalı**

6	Yalnızca getter yazın	Hiçbir alan için `setter` yazmayın; `setDeger()` **olmamalıdır**
7	`toString()` ezin	`TRB-01 = -12.0 C  (kabul=2, red=2)` biçiminde çıktı vermeli

Tablo 3.3 — Görev 2 gereksinimleri

Ardından `Gorev2_SensorDemo.java` adında bir sürücü program yazın; sınıfı hem geçerli hem geçersiz ölçümlerle sınasın.

Beklenen Çıktı

```
▸ TERMINAL — Gorev2_SensorDemo — geçerli ve geçersiz ölçümler
Baslangic : TRB-01 = 0.0 C  (kabul=0, red=0)

--- Gecerli olcumler ---
  olc(21.5)  -> true
  olc(-12.0) -> true
  Durum: TRB-01 = -12.0 C  (kabul=2, red=0)

--- Aralik disi olcumler ---
  RED  TRB-01: 999.0 C araligin disinda [-50.0, 150.0]
  olc(999.0) -> false
  RED  TRB-01: -80.0 C araligin disinda [-50.0, 150.0]
  olc(-80.0) -> false
```

SIK YAPILAN HATA

En sık yapılan hata, doğrulamayı `main` metodunun içine yazmaktır: `if (deger < -50) ... else s.olc(deger);` Bu, kuralı sınıfın **dışına** taşır. Kural sınıfın içinde olmalıdır; çağıran kod hiçbir denetim yapmamalıdır.

İPUCU

`setter` yazıp yazmama ölçütü: *"Bu alanı dışarıdan keyfî bir değere ayarlamak anlamlı bir iş işlemi midir?"* Sensör için yanıt hayırdır — sensör **ölçüm yapar**, değeri elle ayarlanmaz. Bu yüzden metot adı `setDeger` değil `olc` olmalıdır.

Kontrol Soruları

1. `kod` alanını `private` yapmak yetmez, ayrıca `final` de yaptık. `final` olmasaydı hangi kapı açık kalırdı?
2. `olc()` metodunun `boolean` döndürmesi, `void` döndürmesine göre çağıran tarafa ne kazandırır?
3. `EN_DUSUK` ve `EN_YUKSEK` sabitlerini `public` yapmak kapsüllemeyi ihlal eder mi? Gerekçelendirin.

Görev 3 — Miras: Kisi ve Ogrenci

Süre: 25 dakika · Puan: 20 · Zorluk: ●●○

Amaç

İki seviyeli bir miras hiyerarşisi kurmak; hangi üyelerin devralındığını ve `super(...)` çağrısının nereye yazıldığını uygulamalı olarak görmek.

Görev Tanımı

Bir okul kayıt sistemi için `Kisi` üst sınıfını ve ondan türeyen `Ogrenci` alt sınıfını yazın. `Ogrenci` sınıfı, `Kisi` sınıfında zaten var olan hiçbir üyeyi **tekrar yazmamalıdır**.

Sınıf	Alanlar	Metotlar
`Kisi`	`ad`, `tcNo`, `dogumYili` (tümü `private final`)	`getAd()`, `getTcNo()`, `getDogumYili()`, `maskeliTc()`, `yas()`, `tanit()`
`Ogrenci extends Kisi`	`numara` (`private final`), `ortalama` (`private`)	`getNumara()`, `getOrtalama()`, `setOrtalama()`, `onurListesinde()`, `tanit()` (**ezme**)

Tablo 3.4 — Görev 3 sınıf tasarımı

Gereksinimler

- `Kisi` yapıcısı üç parametre alsın; geçersiz değerler için makul varsayılan kullansın (boş ad → `isimsiz`, 11 haneli olmayan TC → `00000000000`).
- `maskeliTc()` metodu TC numarasının ortasını gizlesin: `987*****109`.
- `yas()` metodu `2026 - dogumYili` döndürsün.
- `Ogrenci` yapıcısı **ilk satırında** `super(...)` çağırsın.
- `setOrtalama()` yalnızca `0.0`–`4.0` aralığını kabul etsin; dışındakini reddedip uyarı yazsın.
- `onurListesinde()` metodu ortalama `3.0` ve üzeriyse `true` döndürsün.
- `Ogrenci.tanit()` metodunu `@Override` ile ezin; çıktısında öğrenci numarası, ortalama ve onur listesi durumu görünsün.
- `Gorev3_MirasDemo.java` sürücü programı, `Ogrenci` nesnesinin `Kisi`'den devraldığı üç üyeyi (`getAd()`, `maskeliTc()`, `yas()`) çağırarak devralmayı kanıtlasın.

Beklenen Çıktı

```
▸ TERMINAL — Gorev3_MirasDemo — devralınan üyeler
--- Devralinan uyeler ---
  Zeynep Kaya    123*****901  yas=27  [Kisi]
  Emre Demir     987*****109  yas=21  [Ogrenci no=376512 ort=3.42 *onur*]
```

```
--- Ogrenci, Kisi'den NELERI devraldi? ---
  o.getAd()      = Emre Demir
  o.maskeliTc()  = 987*****109
  o.yas()        = 21
  (bu uc uye Ogrenci.java icinde HIC yazilmadi)
```

```
▸ TERMINAL — Gorev3_MirasDemo — IS-A ilişkisi ve kapsülleme
--- IS-A iliskisi ---
  o instanceof Ogrenci = true
  o instanceof Kisi    = true
  k instanceof Ogrenci = false

--- Kapsulleme mirasi bozmuyor ---
  Uyari: ortalama 0.0-4.0 araliginda olmalidir; 0.0 kullanildi.
  ortalama = 0.0
  ortalama = 3.75
  Emre Demir     987*****109  yas=21  [Ogrenci no=376512 ort=3.75 *onur*]
```

> **DİKKAT**
>
> `Ogrenci.java` dosyasında `getAd()`, `maskeliTc()` veya `yas()` metotlarından herhangi biri yazılmışsa görev **eksik** sayılır. Mirasın amacı tam olarak bunları tekrar yazmamaktır.

Kontrol Soruları

1. `Ogrenci` sınıfı `Kisi` sınıfındaki `private String ad;` alanına doğrudan erişebilir mi? `getAd()` metoduna erişebilir mi? İkisi arasındaki farkı açıklayın.
2. `super(ad, tcNo, dogumYili);` çağrısını yapıcının ikinci satırına yazarsanız ne olur?
3. `k instanceof Ogrenci` ifadesi neden `false`, `o instanceof Kisi` ifadesi neden `true` döner?

Görev 4 — Çok Biçimlilik ve Güvenli Dönüşüm

Süre: 20 dakika · Puan: 20 · Zorluk: ●●○

Amaç

Üst tür referansı içeren bir dizi üzerinde tek döngüyle çok biçimli işlem yapmak; `instanceof` ile güvenli aşağı dönüşüm yazmak.

Görev Tanımı

Görev 3'teki hiyerarşiye `Personel` adında ikinci bir alt sınıf ekleyin, ardından üç türü birlikte içeren bir `Kisi[]` dizisi üzerinde çalışın.

Gereksinimler

#	Gereksinim	Ölçüt
1	`Personel extends Kisi` sınıfını yazın	`unvan` ve `hizmetYili` alanları; `kidemKatsayisi()` metodu
2	`kidemKatsayisi()` metodu	`1.0 + hizmetYili * 0.05` döndürsün
3	`Personel.tanit()` metodunu ezin	`@Override` ile; unvan ve kıdem katsayısı görünsün
4	Dört öğeli `Kisi[]` dizisi kurun	Bir `Kisi`, iki `Ogrenci`, bir `Personel`
5	`listele(Kisi[])` metodu yazın	İçinde `Ogrenci` ve `Personel` sözcükleri **geçmesin**; `switch` ve `if` olmasın
6	Onur listesindekileri sayın	`instanceof` ile sınayıp güvenli dönüşüm yapın
7	Statik ve dinamik türü yazdırın	`getClass().getSimpleName()` kullanın

Tablo 3.5 — Görev 4 gereksinimleri

Beklenen Çıktı

```
▸ TERMINAL — Gorev4_CokBicimlilikDemo — dinamik bağlama ve statik/dinamik tür
--- 1) Dinamik baglama ---
  Zeynep Kaya    123*****901  yas=27  [Kisi]
  Emre Demir     987*****109  yas=21  [Ogrenci no=376512 ort=3.42 *onur*]
  Sena Ak        111*****344  yas=22  [Ogrenci no=376498 ort=2.10]
  Murat Ozturk   555*****788  yas=41  [Personel Teknisyen kidem=1.60]

--- 2) Statik tur / dinamik tur ---
  Bildirim turu (statik) : Kisi
  Gercek tur   (dinamik) : Ogrenci

--- 3) instanceof ile guvenli asagi donusum ---
```

```
▸ TERMINAL — Gorev4_CokBicimlilikDemo — güvenli dönüşüm
```

```
  Onur listesindeki ogrenci sayisi: 1

--- 4) Ust tur referansi neyi GOREMEZ? ---
  ref.getOrtalama();  DERLENMEZ - Kisi'de boyle bir uye yok
  Cozum: once instanceof, sonra donusum.
```

SIK YAPILAN HATA

`Ogrenci o = (Ogrenci) k;` satırını `instanceof` sınaması olmadan yazarsanız, `k` bir `Personel` olduğunda program `ClassCastException` fırlatarak çöker. Dönüşümden önce **her zaman** sınayın.

İPUCU

`listele()` metodunun içinde hiçbir alt sınıf adı geçmemelidir. Bunu sınamanın kolay yolu: metodu yazdıktan sonra dosyada `Ctrl+F` ile `Ogrenci` ve `Personel` sözcüklerini arayın; `listele()` gövdesinde bulunmamalıdır.

Kontrol Soruları

1. `Kisi ref = kayitlar[1];` bildiriminden sonra `ref.getOrtalama()` çağrısı neden derlenmez? Nesne gerçekte bir `Ogrenci` olmasına rağmen.
2. `listele()` metoduna beşinci bir tür (örneğin `Mezun`) eklenirse bu metotta kaç satır değişir? Neden?
3. `getClass().getSimpleName()` çağrısı derleme zamanında mı çalışma zamanında mı sonuç üretir? Yanıtınızı çıktıyla gerekçelendirin.

Görev 5 — İleri Düzey: Mini Kütüphane (Bonus)

Süre: 20 dakika · Puan: 20 (bonus) · Zorluk: ●●●

Amaç

Dört ilkeyi **aynı anda** kullanan küçük ama eksiksiz bir sistem tasarlamak.

Görev Tanımı

Bir fakülte kütüphanesi için `Gorev5_KutuphaneDemo.java` dosyasında aşağıdaki yapıyı kurun. Tüm sınıflar tek dosyada olabilir; yalnızca bir tanesi `public` olmalıdır.

Tür	Rol	Hangi ilke?
`abstract class Yayin`	Ortak çekirdek: `kod`, `baslik`, `oduncta`	Kapsülleme + Soyutlama + Miras
`interface Oduncalinabilir`	`int oduncGunu();` sözleşmesi	Soyutlama
`class Kitap extends Yayin implements Oduncalinabilir`	Yazar ve sayfa sayısı; 15 gün ödünç	Miras + Soyutlama
`class Dergi extends Yayin`	Yıl ve sayı; **ödünç verilemez**	Miras
`class Kutuphane`	Raf dizisi ve ödünç verme kuralları	Kapsülleme + Çok biçimlilik

Tablo 3.6 — Görev 5 tür tasarımı

Gereksinimler

- `Yayin` sınıfı soyut olsun ve `abstract String etiket();` sözleşmesini tanımlasın. `toString()` metodu bu sözleşmeyi çağırsın.
- `oduncta` alanı `private` olsun; dışarıdan doğrudan değiştirilemesin.
- `Kutuphane.oduncVer(String kod)` metodu **dört** durumu ayrı ayrı ele alsın: kayıt yok · tür ödünç verilemez · zaten ödünçte · başarılı.
- Ödünç verilebilirlik denetimi `instanceof Oduncalinabilir` ile yapılsın; sınıf adı (`Kitap`) **kullanılmasın**.
- `Kutuphane.listele()` metodu tek döngüyle her yayını kendi biçiminde bassın.
- `raf` dizisi dışarıya **hiç verilmesin**; istatistikler metotlarla sunulsun.
- Her ilkenin kullanıldığı satırı `// KAPSULLEME`, `// SOYUTLAMA`, `// MIRAS`, `// COK BICIMLILIK` yorumlarıyla işaretleyin.

Beklenen Çıktı

```
▸ TERMINAL — Gorev5_KutuphaneDemo — başlangıç durumu ve ödünç verme
--- 1) Baslangic durumu (COK BICIMLILIK) ---
  Of Teknoloji Fakultesi Kutuphanesi
  978-975-01   Nesne Yonelimli Programlama  [RAFTA]   Kitap  Deitel     1200 sf.
  978-975-02   Java'nin Temelleri           [RAFTA]   Kitap  Horstmann   860 sf.
  1300-1234    Bilim ve Teknik              [RAFTA]   Dergi  2026/8  (odunc verilemez)
```

```
--- 2) Odunc verme (KAPSULLEME: kural tek yerde) ---
  OK   978-975-01: 15 gun icin verildi.
  RED  978-975-01: zaten oduncta.
  RED  1300-1234: bu tur odunc verilemez.
  RED  YOK-999: kayit bulunamadi.
```

```
▸ TERMINAL — Gorev5_KutuphaneDemo — iade ve istatistik
--- 4) Iade ve istatistik (SOYUTLAMA) ---
  OK   978-975-01: iade alindi.
  Raftaki yayin sayisi     : 3
  Odunc verilebilir olanlar: 2
  Su an oduncta            : 0
```

> **İPUCU**
>
> `Dergi` sınıfı `Oduncalinabilir` arayüzünü gerçeklemez. Bu, "dergi ödünç verilemez" kuralının **tür sisteminde** ifade edilmesidir; bir `if` yazmaya gerek kalmaz. Tasarımın kuralı taşıması, kodun kuralı taşımasından daha güçlüdür.

Kontrol Soruları

1. `Dergi` sınıfına `oduncGunu()` metodu yazarsanız ne olur? Dergi ödünç verilebilir hâle gelir mi?
2. `raf` dizisini `public Yayin[] getRaf()` ile dışarıya verseydik hangi kapsülleme ihlali doğardı?
3. `listele()` metodu `Kitap` ve `Dergi` sözcüklerini hiç geçirmediği hâlde farklı biçimlerde çıktı basıyor. Bunu sağlayan mekanizma nedir?

Değerlendirme ve Teslim

Puanlama Ölçütleri

Görev	Ölçüt	Puan
Görev 1	Dört bölgenin ilkesi doğru tespit edilmiş	10
Görev 1	Gerekçelerde anahtar sözcük/satır belirtilmiş	10
Görev 2	Tüm alanlar `private`; `final` olanlar doğru seçilmiş	8
Görev 2	Aralık denetimi `olc()` içinde; setter yazılmamış	8
Görev 2	Sayaçlar doğru çalışıyor, `toString()` biçimli	4
Görev 3	`extends` + `super(...)` doğru kullanılmış	8
Görev 3	Devralınan üyeler tekrar yazılmamış	7
Görev 3	`tanit()` `@Override` ile ezilmiş	5
Görev 4	`Personel` sınıfı ve ezme doğru	6
Görev 4	`listele()` içinde alt sınıf adı geçmiyor	8
Görev 4	`instanceof` ile güvenli dönüşüm yapılmış	6
Görev 5 (bonus)	Soyut sınıf + arayüz doğru kurulmuş	8
Görev 5 (bonus)	Dört durum ayrı ayrı ele alınmış	7
Görev 5 (bonus)	İlke yorumları doğru yerleştirilmiş	5

Tablo 3.7 — Görev bazında puan dağılımı (toplam 80 + 20 bonus)

Teslim Öncesi Kontrol Listesi

Teslim Öncesi Kontrol Listesi

- [ ] Tüm `.java` dosyaları **hatasız** derleniyor (`javac -d bin src/*.java`).
- [ ] Her dosyanın başında ad, soyad, numara ve görev numarası var.
- [ ] Kod ve yorumlarda Türkçe karakter kullanılmamış.
- [ ] Tüm alanlar `private`; gereksiz hiçbir `setter` yazılmamış.
- [ ] Ezilen her metotta `@Override` işaretleyicisi var.

`Ogrenci.java` içinde `getAd()`, `maskeliTc()`, `yas()` **yok**.

`listele()` metodunda alt sınıf adı, `switch` veya `instanceof` **yok**.

Aşağı dönüşümlerin tamamı `instanceof` sınamasından sonra yapılmış.

Kontrol sorularının yanıtları ilgili dosyaların sonunda yorum olarak var.

Klasör `Hafta03_<Numara>_<AdSoyad>.zip` biçiminde sıkıştırılmış.

Sık Karşılaşılan Hatalar

Belirti	Olası neden	Çözüm
`Sekil is abstract; cannot be instantiated`	Soyut sınıftan `new` ile nesne üretilmeye çalışıldı	Somut alt sınıf kullanın: `new Daire(...)`
`Ogrenci is not abstract and does not override abstract method ...`	Soyut metot doldurulmadı	Eksik metodu yazın veya sınıfı `abstract` bildirin
`constructor Kisi in class Kisi cannot be applied to given types`	`super(...)` çağrısı eksik veya parametreleri uyuşmuyor	Yapıcının **ilk satırında** doğru `super(...)` yazın
`cannot find symbol: method getOrtalama()`	Üst tür referansıyla alt tür üyesine erişilmeye çalışıldı	`instanceof` ile sınayıp `(Ogrenci)` dönüşümü yapın
`ClassCastException` (çalışma zamanı)	Sınama yapılmadan aşağı dönüşüm yapıldı	Dönüşümden önce `instanceof` sınaması ekleyin
`method does not override or implement a method from a supertype`	`@Override` var ama üst türde eşleşen metot yok	Metot adını ve parametre listesini üst sınıfla eşitleyin
Ezdiğiniz metot çalışmıyor, üst sınıfınki çalışıyor	Metot adı veya parametre listesi farklı; ezme yerine aşırı yükleme oldu	`@Override` ekleyin; derleyici sizi uyaracaktır

Tablo 3.8 — Laboratuvarda sık karşılaşılan sorunlar

> **SON SÖZ**
>
> Görevleri süre içinde tamamlayamazsanız panik yapmayın. Tamamladığınız kadarını teslim edin ve eksik kısmı bir sonraki laboratuvardan önce bitirin. Bu hafta amaç hızlı kod yazmak değil, **dört ilkeyi çalışan bir kod üzerinde tanımaktır**. Görev 5 bonustur; Görev 1–4'ü sağlam bitirmek her zaman önceliklidir.